

Introduction to Automata Theory (2nd Part)

Daibik Barik, Shankha Suvra Dam

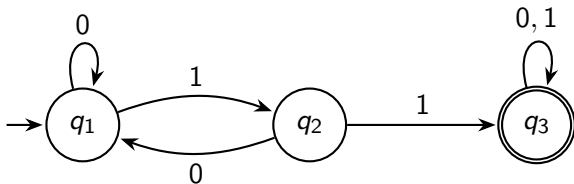
22 May 2026

What's a Finite Automaton?

A **Deterministic** (will see this later!) **Finite Automaton** (DFA) is rigorously defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- ▶ Q is a finite set of *states*.
- ▶ Σ is a finite set called the *alphabet* (the available inputs).
- ▶ $\delta : Q \times \Sigma \rightarrow Q$ is the *transition function* (the rules).
- ▶ $q_0 \in Q$ is the *start state*.
- ▶ $F \subseteq Q$ is the set of *accept states* (the goal condition).

Anatomy of an Automaton (M_1)



This machine M_1 recognizes strings containing the substring "11".

What's a Regular Language?

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a deterministic finite automaton.

M **accepts** a string $w = w_1 w_2 \dots w_n$ if a sequence of states $r_0, r_1, \dots, r_n$ in Q exists such that:

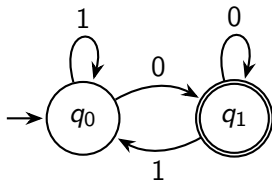
1. $r_0 = q_0$ (*We start at the beginning*)
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ for $i = 0, \dots, n - 1$ (*We follow the rules*)
3. $r_n \in F$ (*We land on an accept state*)

The **language of machine** M , denoted $L(M)$, is the set of all strings it accepts.

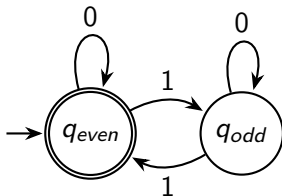
Definition: A language is called a **Regular Language** if some finite automaton recognizes it.

Examples of Regular Languages

Example A: $L_1 = \{w \mid w \text{ ends with a } 0\}$ over $\Sigma = \{0, 1\}$

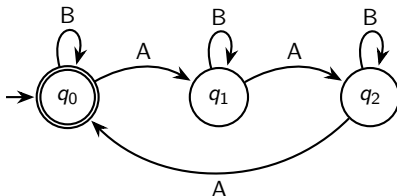


Example B: $L_2 = \{w \mid w \text{ has an even number of 1s}\}$

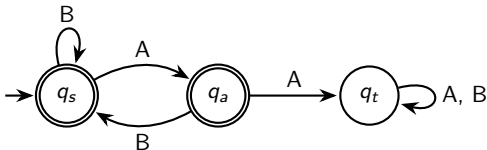


Solutions to the Jewel Puzzles

Puzzle 1: $L_1 = \{w \mid \text{number of A's is a multiple of 3}\}$



Puzzle 2: $L_2 = \{w \mid w \text{ never contains consecutive A's}\}$



Why do we care IF a language is Regular?

Knowing whether a language is Regular tells us exactly which tools we are allowed to use.

- ▶ *Why use a cannon to kill a mosquito?:*

If a problem is *not* regular (like checking $A = B$, or matching nested parentheses), no Regular Expression in the world will ever solve it.

- ▶ **Choosing the Right Machine:**

- ▶ **If Regular:** It is incredibly fast and cheap to compute (zero working memory needed).
- ▶ **If Not Regular:** You immediately know you must “upgrade” your machine (e.g., adding a memory stack, which costs more computational resources).

- ▶ *fun fact:* This boundary is exactly why software engineers who try to parse nested HTML tags using RegEx always fail. HTML is not a regular language!

How do we know if a language is Regular?

If a language is simple, we can just draw a DFA. But what if the language has highly complex rules?

Drawing massive diagrams from scratch is inefficient and prone to error.

Instead of building one giant machine, we build small ones and combine them mathematically. To do this, we must first define **Regular Operations**.

What is a Regular Operation?

In arithmetic, we use operations like $+$ or $\times$ to build new numbers from old ones.

In Automata Theory, we use **Regular Operations** to build new, complex languages out of simple ones.

- ▶ We treat languages as **sets of strings**.
- ▶ Regular operations allow us to combine these sets mathematically.
- ▶ **Example:** Combining a machine that checks for “@” with a machine that checks for “gmail.com” to validate a gmail email address.

The Three Basic Operations

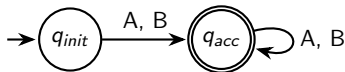
Let A and B be languages. We define:

1. **Union** ($A \cup B$): The set of strings that are in A **OR** in B .
2. **Concatenation** ($A \circ B$): A string from A followed by a string from B .
3. **Star** (A^*): Zero or more repetitions of strings from A .
 - *Example:* If $A = \{0\}$, then $A^* = \{\epsilon, 0, 00, 000, \dots\}$

Regular Operation Examples: Union

Let $L_1 = \{w \mid w \text{ starts with A}\}$ and $L_2 = \{w \mid w \text{ starts with B}\}$

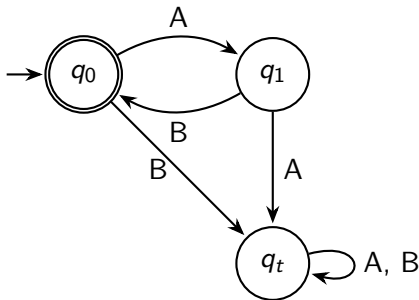
The Union $L_1 \cup L_2$ accepts strings starting with **either** A or B.



Note: If L_1 and L_2 are regular, we can always build a "Product Automaton" that runs both machines in parallel to recognize the union.

Regular Operation Examples: Star

Let $L = \{AB\}$. Then $L^* = \{\epsilon, AB, ABAB, ABABAB, \dots\}$



Star Operation allows for repetition. This is how we define "loops" in patterns without drawing them as separate states for every possible count.

Jewel Puzzles as Operations

Recall our first two regular puzzles:

- ▶ L_1 : Multiple of 3 A's
- ▶ L_2 : No consecutive A's

Because Regular Languages are **closed under Union and Concatenation**:

1. We can describe a machine for: "*A string with a multiple of 3 A's **OR** no consecutive A's.*" ($L_1 \cup L_2$)
2. We can describe: "*A string that starts with a multiple of 3 A's and **ends** with no consecutive A's.*" ($L_1 \circ L_2$)

Closure guarantees that these combined "super-rules" are still Regular.

Why do we care about these operations?

- ▶ **Modularity:** It is easier to design a machine for a small task (e.g., “Contains an A”) and combine it with another (e.g., “Contains a B”) than to draw one giant machine from scratch.
- ▶ **Complexity:** They allow us to describe complicated patterns (like a valid email address or a phone number) as a combination of simpler rules.
- ▶ **Relevance to our example:** If we can prove that A and B are Regular, and we union them, is the result still Regular?

What is “Closure”?

A set is **closed** under an operation if applying that operation to members of the set always results in a member of that same set.

- ▶ **Analogy:** The set of Integers is closed under Addition.
($5 + 3 = 8$. You won't suddenly get some non- $\mathbb{Z}$).
- ▶ **In our world:** The class of Regular Languages is **closed** under Union, Concatenation, and Star.
- ▶ **The Meaning:** If I combine two Regular Languages using these rules, the result **can always be represented by a DFA**.

How to know if a language is regular?

To check if a language is Regular, we look for:

1. **Existence of a DFA:** Can you draw a finite state diagram for it?
2. **Existence of a RegEx:** Can the language be expressed using Union, Concatenation, and Star?
3. **Memory Requirement:** Does the machine need to track a value that could grow infinitely? (If yes, it is likely not regular).

Showing non-regularity of the last Jewel Puzzle

Theorem: $L = \{w \mid \text{number of A's} = \text{number of B's}\}$ is not regular. **Proof:**

1. Suppose L is regular. Then there exists a DFA M with k states.
2. Consider the strings $s_1 = A^1, s_2 = A^2, \dots, s_{k+1} = A^{k+1}$.
3. Since there are $k + 1$ strings but only k states, **two strings must land in the same state** (say s_i and s_j where $i \neq j$).
4. If M is in the same state after A^i and A^j , it must behave identically for any remaining input.
5. Append B^i to both. M must either accept both or reject both.
6. But $A^i B^i \in L$ (Accept) and $A^j B^i \notin L$ (Reject).

Conclusion: M cannot distinguish i from j . To "count" exactly, a machine needs a different state for every possible number of A's.
Finite states $\implies$ cannot count to infinity.

Side Tangent: Regular Expressions

- ▶ **Definition:** A text-based shorthand for describing regular languages.
- ▶ **Equivalence:** Any language that can be described by a DFA can be written as a Regular Expression (Regex).
- ▶ Used in almost every programming language to find, match, or validate strings (e.g., email or phone number formats).
- ▶ Explore some of them at: regex101.com

RegEx in Action: Demonstrations

Example 1: Words starting with 'H'

RegEx Code: `H[a-z]*`

Matches in text:

Hello there! Have a Happy day. How are you?

Example 2: The Union Operation (OR)

RegEx Code: `cat|dog`

Matches in text:

I have a cat and my neighbor has a dog, but no bird.

Under the hood, your programming language converts `cat|dog` directly into a DFA with two branching paths from the start state before it ever scans the text.

Extra Stuff: Pushing Operations to the Limit

Later, we will study machines with higher memory (like PDAs with Stacks). But why do we need them if Regular Languages are so robust?

Regular languages survive almost everything, they are closed under Reversal, Differences, and String Quotients. (refer to the problemset to be sent!)

But what happens if we scale an operation to infinity?

- ▶ **The Infinite Union:** Instead of uniting two languages ($L_1 \cup L_2$), what if we unite an infinite number of them?
- ▶ Mathematically written as: $\bigcup_{i=0}^{\infty} L_i$

Extra Stuff: Lack of Closure under Infinite Union

Theorem: Regular Languages are **NOT** closed under Infinite Union.

Proof:

1. Let $L_i = \{A^i B^i\}$ for a specific, fixed integer i (e.g., $L_3 = \{AAABBB\}$).
2. Any finite language is strictly **Regular**. (You can just draw a straight line of states to accept exactly one string).
3. Now, take the infinite union of all these individual regular languages:

$$L = L_0 \cup L_1 \cup L_2 \cup \dots = \bigcup_{i=0}^{\infty} \{A^i B^i\}$$

4. This results in $L = \{w \mid \#A = \#B\}$.
5. As we just proved, this final language is **Not Regular!**

Takeaway: Finite machines cannot survive infinite scaling. To process languages built from infinite unions, we must upgrade our hardware to machines with infinite memory stacks.

Extra Stuff: Closure under Complement Proof

Theorem: If L is regular, then its complement $\bar{L} = \Sigma^* \setminus L$ is also regular.

Proof:

1. Since L is regular, there exists a DFA $M = (Q, \Sigma, \delta, q_0, F)$ that recognizes it.
2. We construct a new DFA $M' = (Q, \Sigma, \delta, q_0, Q \setminus F)$.
3. M' is identical to M except its accept states are the **original non-accept states**.
4. A string w is accepted by M' iff it ends in a state in $Q \setminus F$, which is true iff $w \notin L$.

Conclusion: Swapping accept/reject states proves closure under complement.

Extra Stuff: Closure under Intersection Proof

Theorem: If L_1 and L_2 are regular, then $L_1 \cap L_2$ is regular.

Proof (Product Construction):

1. Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.
2. Construct $M = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$.
3. The new transition is $\delta((r, s), a) = (\delta_1(r, a), \delta_2(s, a))$.
4. M accepts iff **both** M_1 and M_2 reach an accept state.

Conclusion: We can simulate two DFAs in parallel using one DFA.